

Practical – 5

Practical 5 AIM:

Perform Create, Read, Update, and Delete (CRUD) operations using MongoDB.

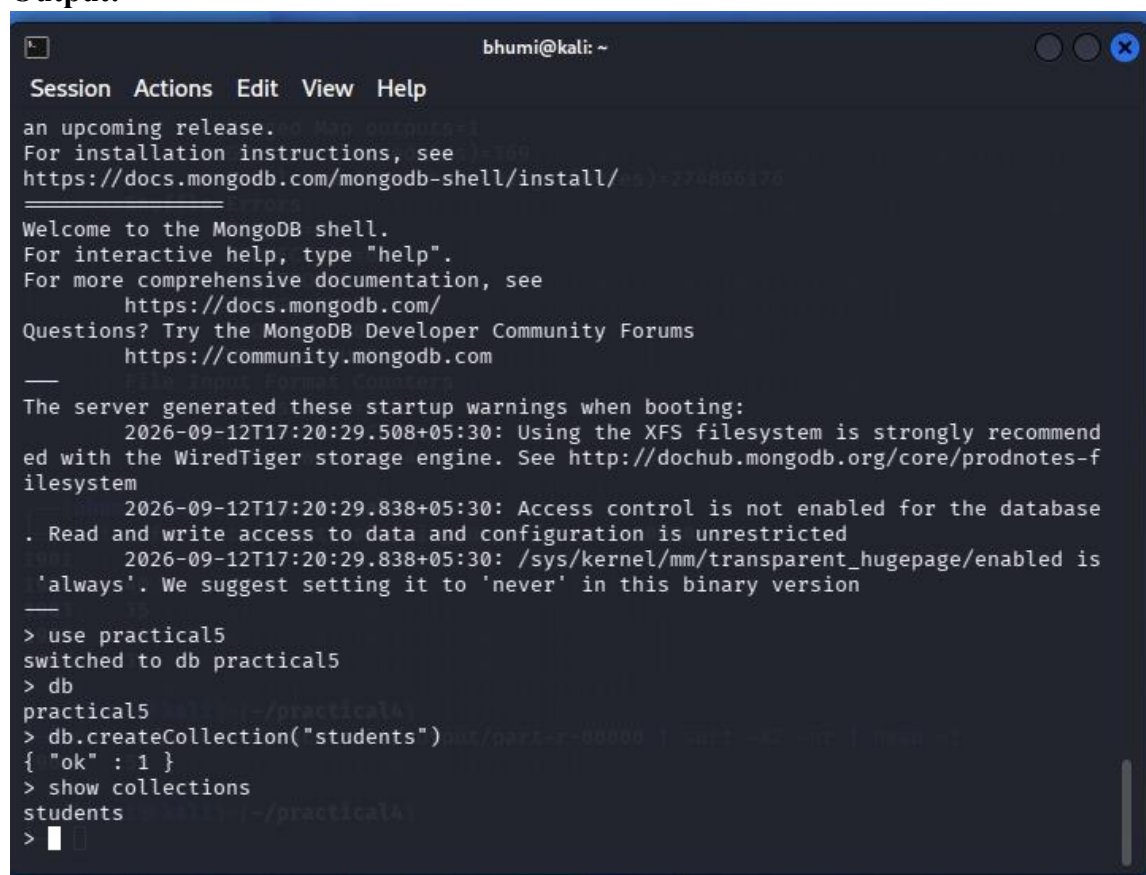
Problem Definition :

Perform Create, Read, Update, and Delete (CRUD) operations using MongoDB. Students will work with a NoSQL document database to understand how unstructured and semi-structured data is managed at scale in Big Data environments.

Tasks:

1. Install and configure MongoDB.
2. Create a database and collection.
3. Insert documents (single and multiple).
4. Read/Query documents using various filters.
5. Update existing documents.
6. Delete documents and collections.
7. Use MongoDB aggregation pipeline for data analysis.

Output:



```
bhumi@kali: ~  
Session Actions Edit View Help  
an upcoming release. For installation instructions, see https://docs.mongodb.com/mongodb-shell/install/  
Welcome to the MongoDB shell.  
For interactive help, type "help".  
For more comprehensive documentation, see https://docs.mongodb.com/  
Questions? Try the MongoDB Developer Community Forums https://community.mongodb.com  
The server generated these startup warnings when booting:  
2026-09-12T17:20:29.508+05:30: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem  
2026-09-12T17:20:29.838+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted  
2026-09-12T17:20:29.838+05:30: /sys/kernel/mm/transparent_hugepage/enabled is 'always'. We suggest setting it to 'never' in this binary version  
> use practical5  
switched to db practical5  
> db  
practical5  
> db.createCollection("students")  
{ "ok" : 1 }  
> show collections  
students  
>
```

```

bhumi@kali: ~
Session Actions Edit View Help
> db.students.find({ branch: "Computer Engineering" }).pretty()
{
  "_id" : ObjectId("6aa53dad945bc47c785d0f0b"),
  "student_id" : 101,
  "name" : "Bhumi Vora",
  "age" : 20,
  "branch" : "Computer Engineering",
  "semester" : 6,
  "city" : "Anand"
}
{
  "_id" : ObjectId("6aa53f87dc190b80fdbec7"),
  "student_id" : 102,
  "name" : "Aarav Mehta",
  "age" : 21,
  "branch" : "Computer Engineering",
  "semester" : 6,
  "city" : "Kolkata"
}
{
  "_id" : ObjectId("6aa53f87dc190b80fdbec9"),
  "student_id" : 104,
  "name" : "Dev Kumar",
  "age" : 23,
  "branch" : "Computer Engineering",
  "semester" : 6,
  "city" : "Delhi"
}
>

```

```

bhumi@kali: ~
Session Actions Edit View Help
  "student_id" : 104,
  "name" : "Dev Kumar",
  "age" : 23,
  "branch" : "Computer Engineering",
  "semester" : 6,
  "city" : "Delhi"
}
> db.students.deleteMany({ age: { $gt: 21 } })
{ "acknowledged" : true, "deletedCount" : 1 }
> db.students.find().pretty()
{
  "_id" : ObjectId("6aa53dad945bc47c785d0f0b"),
  "student_id" : 101,
  "name" : "Bhumi Vora",
  "age" : 20,
  "branch" : "Computer Engineering",
  "semester" : 6,
  "city" : "Anand"
}
{
  "_id" : ObjectId("6aa53f87dc190b80fdbec7"),
  "student_id" : 102,
  "name" : "Aarav Mehta",
  "age" : 21,
  "branch" : "Computer Engineering",
  "semester" : 6,
  "city" : "Kolkata"
}
>

```

```

bhumid@kali: ~
Session Actions Edit View Help

... },      Merged Map outputs=1
... {
...   student_id: 107, red heap usage (bytes)=274866176
...   $type: "Meera Rao",
...   age: 21,
...   branch: "Computer Engineering",
...   semester: 6,
...   city: "Bhopal"
... }
... ]
{
  File Input Format Counters:
    "acknowledged": true,
    "insertedIds" : [
      ObjectId("6aa540cdc190b80fdb1ecf"),
      ObjectId("6aa540cdc190b80fdb1ed0"),
      ObjectId("6aa540cdc190b80fdb1ed1")
    ]
}

> db.students.aggregate([
... {
...   $group: {
...     _id: "$branch",
...     total_students: { $sum: 1 }
...   }
... })
{ "_id" : "Computer Engineering", "total_students" : 3 }
{ "_id" : "Information Technology", "total_students" : 2 }
>

```

```

bhumid@kali: ~
Session Actions Edit View Help
]
}
> db.students.aggregate([
...   {
...     $group: {
...       _id: "$branch",
...       total_students: { $sum: 1 }
...     }
...   }
... ])
{ "_id" : "Computer Engineering", "total_students" : 3 }
{ "_id" : "Information Technology", "total_students" : 2 }
> db.temp_students.insertOne({
...   student_id: 999,
...   name: "Test Student",
...   branch: "Computer Engineering"
... })
{
  "acknowledged" : true,
  "insertedId" : ObjectId("6aa540ffdc190b80fdbc1ed3")
}
> show collections
students
temp_students
> db.temp_students.drop()
true
> show collections
students
>

```

Key Questions / Analysis**Q1. What is MongoDB and how does it differ from relational databases?**

MongoDB is a **NoSQL, document-oriented database** that stores data in flexible JSON-like documents. Unlike relational databases such as MySQL, MongoDB does not primarily use fixed tables and rows. It provides a flexible schema and is suitable for large and changing datasets.

Q2. What are CRUD operations and why are they fundamental to database management?

CRUD stands for **Create, Read, Update, and Delete**. These are the four basic operations used to manage data in a database. They allow users to add new data, retrieve existing data, modify data, and remove unwanted data.

Q3. How does MongoDB store data (BSON format)?

MongoDB stores data as **BSON (Binary JSON)** documents. BSON is a binary representation of JSON-like data that supports additional data types such as integers, dates, and ObjectId. Documents are stored inside **collections**.

Q4. What is the aggregation pipeline in MongoDB?

The **Aggregation Pipeline** is a framework used to process and analyze documents through multiple stages. It can perform operations such as **filtering, grouping, sorting, and calculating** values. Common stages include \$match, \$group, \$sort, and \$project.

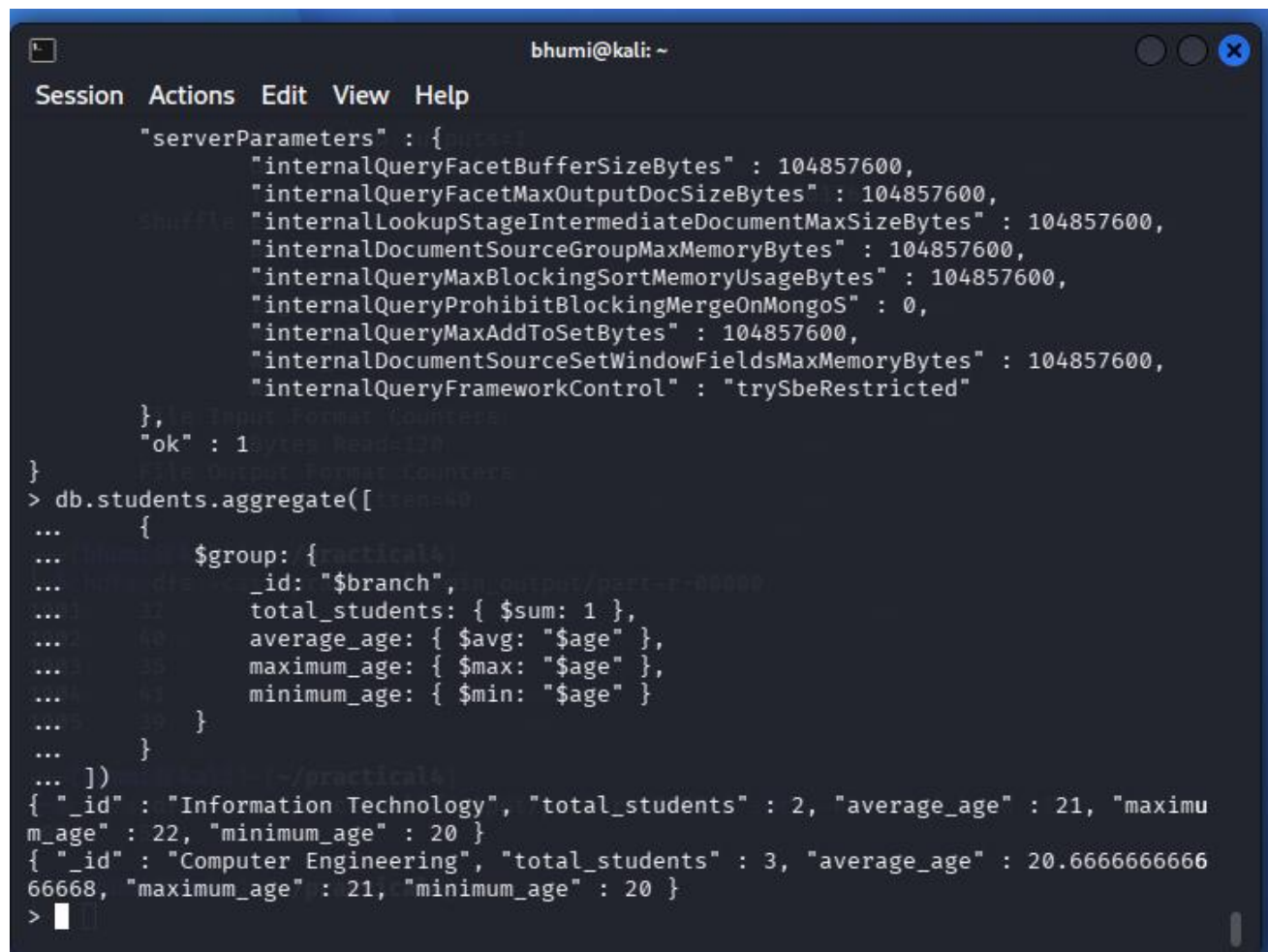
Q5. When should NoSQL databases be preferred over SQL databases?

NoSQL databases should be preferred when applications require **flexible schemas, large-scale data handling, high scalability, and fast processing of unstructured or semi-structured data**. They are commonly useful in Big Data, real-time applications, and rapidly changing datasets.

Supplementary Problems

Create an index on a collection and compare query performance. Use the aggregation pipeline to generate a summary report. Import a JSON dataset into MongoDB and perform analytical queries.

Output:



```

bhumi@kali: ~
Session Actions Edit View Help

"serverParameters" : {
  "internalQueryFacetBufferSizeBytes" : 104857600,
  "internalQueryFacetMaxOutputDocSizeBytes" : 104857600,
  "internalLookupStageIntermediateDocumentMaxSizeBytes" : 104857600,
  "internalDocumentSourceGroupMaxMemoryBytes" : 104857600,
  "internalQueryMaxBlockingSortMemoryUsageBytes" : 104857600,
  "internalQueryProhibitBlockingMergeOnMongoS" : 0,
  "internalQueryMaxAddToSetBytes" : 104857600,
  "internalDocumentSourceSetWindowFieldsMaxMemoryBytes" : 104857600,
  "internalQueryFrameworkControl" : "trySbeRestricted"
},
"ok" : 1
}
> db.students.aggregate([
  {
    $group: {
      _id: "$branch",
      total_students: { $sum: 1 },
      average_age: { $avg: "$age" },
      maximum_age: { $max: "$age" },
      minimum_age: { $min: "$age" }
    }
  })
{ "_id" : "Information Technology", "total_students" : 2, "average_age" : 21, "maximum_age" : 22, "minimum_age" : 20 }
{ "_id" : "Computer Engineering", "total_students" : 3, "average_age" : 20.666666666666668, "maximum_age" : 21, "minimum_age" : 20 }
>

```

Key Skills to be Addressed

NoSQL Database | CRUD Operations | Query & Filtering | Indexing | Query Optimization | Aggregation | JSON Data Import | Data Analysis

Tools /Technology

MongoDB 7.0.14 | Mongo Shell (mongo) | mongoimport | JSON | Kali Linux | Aggregation Pipeline | MongoDB Indexing